SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I jeshyapriya.R(192524453) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
jeshyapriya.R
(192524453)

Question 1: AI-Powered Drones for Emergency Medicine Delivery

i. Explain how Greedy Best-First Search behaves in this scenario. Discuss its strengths and weaknesses.

Greedy Best-First Search is an informed search algorithm that selects the path that appears closest to the destination based on a heuristic function.

The heuristic function is:

$$f(n) = h(n)$$

Where:

- $h(n)$= estimated straight-line distance from the current location to the destination.

Working:

1. The drone identifies its current location and destination.
2. It calculates the estimated distance to the destination.
3. It selects the location with the lowest heuristic value.
4. It continues searching until it reaches the destination.

Strengths:

- Fast decision-making.
- Requires less computation in many cases.
- Useful when emergency medicines must be delivered quickly.

Weaknesses:

- May select a route with blocked roads.
- Does not consider the actual travel cost.
- May choose a dangerous route due to changing weather.
- Does not guarantee the shortest or safest route.

ii. Explain how A* handles the situation.

A* is an informed search algorithm that considers both the actual cost and the estimated remaining cost.

The evaluation function is:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$= actual cost from the starting point to the current location.
- $h(n)$= estimated cost from the current location to the destination.
- $f(n)$= estimated total cost of the path.

Working:

1. The drone identifies possible routes.
2. It calculates the actual travel cost for each route.
3. It estimates the remaining distance to the destination.
4. It adds the actual cost and estimated cost.
5. It selects the route with the lowest total estimated cost.
6. If a road becomes blocked, the drone updates the route information and searches again.

Advantages:

- Considers both distance and travel cost.
- Can find an optimal route under suitable conditions.
- Can account for terrain and weather when these are represented in the cost model.

iii. Analyze the impact of heuristic accuracy on both algorithms.

The heuristic estimates how far the drone is from its destination.

In Greedy Best-First Search:

- A good heuristic helps the drone move towards the destination quickly.
- An inaccurate heuristic may lead the drone towards blocked or expensive routes.
- It does not guarantee an optimal route even with a good heuristic.

In A:*

- An accurate heuristic improves search efficiency.
- An admissible heuristic does not overestimate the remaining optimal cost.
- With suitable conditions, such as an admissible heuristic and appropriate search implementation, A* can find an optimal path.
- A poor heuristic may increase computation or lead to inefficient exploration.

iv. Discuss how dynamic changes affect search performance.

Dynamic changes include blocked roads, heavy rain, flooding, and changing terrain conditions.

These changes can make the previously selected route unsuitable.

Effects:

1. The drone may need to recalculate its route.
2. Previously calculated travel costs may become incorrect.
3. Repeated searches may increase computational time.
4. New hazards may require the drone to choose a safer route.

The drone should use real-time sensor information to update its map and travel costs. It should then replan whenever necessary.

v. Recommend the most suitable algorithm.

Recommendation: A Search with real-time route replanning.*

A* is suitable because it considers both the actual travel cost and the estimated remaining cost.

Reasons:

- It can find an optimal route under appropriate assumptions.
- It can consider weather and terrain costs.
- It can recalculate routes when roads become blocked.
- It provides a balance between route quality and search efficiency.

For safety, the drone should also consider restricted areas, hazard information, battery limitations, and emergency landing locations.

Conclusion: A* with real-time updates is a suitable choice for emergency medicine delivery because it supports cost-effective routing and adaptation to changing conditions.

Question 2: Smart City Traffic Signal Optimization

Formulate the problem as an optimization problem and explain why traditional search is inefficient.

The objective is to optimize traffic signals to minimize waiting time, fuel consumption, and traffic congestion.

Objective function:

$$\min J = w_1 T + w_2 F + w_3 C$$

Where:

- T= total vehicle waiting time.
- F= fuel consumption.
- C= traffic congestion measure.
- w_1, w_2, w_3= weights representing the importance of each objective.

Constraints:

- Minimum and maximum green-light durations.
- Pedestrian crossing requirements.
- Safe transition between signals.
- Emergency vehicle priority.
- Coordination between nearby intersections.

Traditional search algorithms may be inefficient because the number of possible signal combinations grows rapidly as the number of intersections increases. In addition, traffic conditions change continuously.

i. Explain how Hill Climbing works and identify its limitations.

Hill Climbing is a local search algorithm that repeatedly moves from the current solution to a better neighboring solution.

Working:

1. Start with an initial traffic signal schedule.
2. Calculate its performance using the objective function.
3. Generate neighboring schedules by changing signal timings.
4. Evaluate each neighboring schedule.
5. Select an improved schedule.
6. Repeat until no better neighboring solution is found.

Limitations:

- May get stuck in a local optimum.
- May fail to find the global optimum.
- Performance depends on the initial solution.
- May not adapt quickly enough to sudden traffic changes.

ii. Explain local maxima, plateaus, and ridges.

1. Local Maximum

A solution is better than its immediate neighboring solutions but is not the best overall solution.

Example: A signal schedule reduces congestion at one intersection but increases congestion across the entire road network.

2. Plateau

A flat region exists where neighboring solutions have similar performance.

Example: Several signal schedules produce nearly the same waiting time, so Hill Climbing cannot easily identify a better direction.

3. Ridge

A narrow path of improvement requires coordinated changes in multiple variables.

Example: Two connected intersections may need their green-light timings adjusted together to reduce congestion. Changing only one signal may not improve the result.

iii. Explain how Simulated Annealing or Genetic Algorithms overcome these limitations.

Simulated Annealing

Simulated Annealing sometimes accepts a worse solution to escape a local optimum.

Working:

1. Start with an initial signal schedule.
2. Generate a neighboring solution.
3. Accept the solution if it improves the objective.
4. Sometimes accept a worse solution based on a probability controlled by temperature.
5. Gradually reduce the temperature.
6. Continue until a stopping condition is reached.

This method can escape local optima, although it does not guarantee finding the global optimum within a limited runtime.

Genetic Algorithm
A Genetic Algorithm uses a population of possible solutions.
Its steps are:

1. Generate a population of traffic signal schedules.
2. Evaluate each schedule using a fitness function.
3. Select promising schedules.
4. Apply crossover to combine solutions.
5. Apply mutation to introduce variations.
6. Repeat until a stopping condition is met.

Maintaining multiple candidate solutions can help explore different regions of the search space.
iv. Recommend the best approach.
Recommendation: Genetic Algorithm combined with real-time traffic data.
Reasons:

- Can explore multiple candidate schedules.
- Handles large optimization spaces.
- Can optimize several objectives simultaneously.
- Can adapt to changing traffic patterns by periodically updating its population and fitness evaluations.

For real-time use, the algorithm should have a strict computation limit and preserve traffic safety constraints.
Conclusion: A Genetic Algorithm with real-time traffic data is a suitable approach for optimizing complex urban traffic networks.

Question 3: Autonomous Rover Exploring Mars

i. Explain why this problem requires an online search agent instead of offline search.
An online search agent makes decisions while interacting with an environment it does not fully know.
The Mars rover cannot obtain a complete map of the terrain before exploration. It must discover the environment as it moves.
Reasons:

- The terrain is initially unknown.
- Sensors provide only partial information.
- New obstacles may be discovered during exploration.
- The rover must update its decisions using new observations.

Therefore, online search is more suitable than offline search, which generally plans using a model or map available before execution.
ii. Analyze partial observability and dynamic environments.
Partial Observability
The rover cannot observe every part of its surroundings at once.
Challenges:

- Hidden craters and rocks.
- Limited sensor range.
- Uncertainty in distance measurements.
- Incomplete terrain information.

Dynamic Environment
The environment or the rover's knowledge of it may change during exploration.
Challenges:

- Newly detected obstacles.

- Changes in the estimated terrain conditions.
- Limited battery power.
- Communication delays with Earth.

The rover must use sensor observations to update its knowledge and select safe actions.

iii. Describe how the rover builds and updates its internal model.

The rover maintains an internal representation of the environment.

Steps:

1. **Initial model:** Start with the available map and known information.
2. **Sensor observation:** Collect information using cameras and other sensors.
3. **State estimation:** Estimate its current position and surrounding conditions.
4. **Map update:** Add newly discovered paths and obstacles.
5. **Action selection:** Choose the next movement based on the updated map.
6. **Replanning:** Change the route if a hazard is discovered.
7. **Repeat:** Continue observing and updating while exploring.

This process allows the rover to operate even when its initial knowledge is incomplete.

iv. Compare online search with uninformed and informed search strategies.

Feature	Online Search	Uninformed Search	Informed Search
Knowledge	Learns while acting	Uses available problem information	Uses problem information and a heuristic
Heuristic	Optional	Not required	Usually required
Environment	Suitable for unknown environments	Often assumes a known problem representation	Often assumes a model is available
Adaptability	Can update decisions from new observations	Depends on implementation	Depends on implementation
Example	Rover exploring unknown terrain	Breadth-First Search	A* Search

Online search describes when and how an agent searches while interacting with its environment. Uninformed and informed search describe how the agent chooses which states to explore. These categories are not mutually exclusive.

v. Justify the most suitable approach.

Recommendation: An online search agent combined with heuristic guidance and real-time obstacle avoidance.

Reasons:

- Learns about the terrain during exploration.
- Updates its internal map.
- Can respond to newly detected hazards.
- Uses heuristic information to guide exploration when available.
- Supports safer navigation in uncertain environments.

The rover should also consider battery consumption, communication limitations, and emergency stopping conditions.

Conclusion: Online search with heuristic guidance is suitable because the rover must learn and make decisions while exploring unknown terrain.

Question 4: University Examination Timetable Using CSP

Formulate the problem as a Constraint Satisfaction Problem.
A **Constraint Satisfaction Problem (CSP)** consists of variables, domains, and constraints. The objective is to assign values to all variables while satisfying the required constraints.
i. Define variables, domains, and constraints.
1. Variables
Each examination is represented by a variable.
For example:

$$X = \{E_1, E_2, E_3, E_4, E_5\}$$

Where $E_1, E_2, \ldots, E_5$represent five examinations.
Each variable represents the scheduled time slot and hall assignment for an examination.
2. Domains
The domain contains all possible values that can be assigned to each variable.
For example:

$$D(E_i) = \{(T_1, H_1), (T_1, H_2), (T_2, H_1), (T_2, H_2)\}$$

Here:

- $T_1, T_2$are available time slots.
- $H_1, H_2$are examination halls.

The actual domain depends on hall availability, capacity, faculty availability, and other requirements.
3. Constraints
Hard constraints:

1. **No overlapping examinations:** A student cannot have two examinations at the same time.
2. **Hall capacity:** The assigned hall must accommodate the students taking the examination.
3. **Hall availability:** A hall cannot be assigned to two examinations simultaneously.
4. **Subject order:** If one subject must be completed before another, their time slots must follow that order.
5. **Faculty availability:** Examinations must be scheduled when the required faculty members are available.
6. **Special accommodations:** Students requiring special arrangements must be assigned suitable examination facilities and times.

All hard constraints must be satisfied for a valid timetable.
ii. Explain how backtracking search works.
Backtracking is a search technique that assigns values to variables one at a time and reverses assignments when a constraint is violated.
Steps:

1. Select an examination.
2. Choose a time slot and hall from its domain.
3. Check whether the assignment satisfies all constraints.
4. If valid, continue with the next examination.
5. If no valid assignment exists, return to the previous examination.
6. Change the previous assignment and try another possibility.
7. Continue until a complete valid timetable is found or all possibilities have been explored.

Example:
Suppose Mathematics and Physics have common students.
If Mathematics is scheduled at 9:00 AM, Physics cannot be scheduled at 9:00 AM for those students. The algorithm must select another available time slot for Physics.

iii. Explain how constraint propagation improves efficiency.
Constraint propagation reduces the domains of variables by removing values that cannot satisfy the constraints.

Forward Checking:
After assigning a value to one variable, forward checking removes inconsistent values from the domains of related unassigned variables.

Example:
Suppose Mathematics is scheduled at 9:00 AM.
If Physics shares students with Mathematics, the 9:00 AM slot is removed from Physics's domain.

Advantages:

- Detects conflicts early.
- Reduces unnecessary search.
- Helps avoid repeated invalid assignments.
- Improves backtracking efficiency.

iv. Analyze real-world challenges and recommend the best strategy for scalability.

Challenges:

- Large numbers of students and examinations.
- Limited examination halls.
- Faculty availability conflicts.
- Special accommodation requirements.
- Subject-order restrictions.
- Changes to examination dates or student registrations.

Recommendation: Backtracking Search with constraint propagation, the Minimum Remaining Values (MRV) heuristic, and Degree heuristic.
The MRV heuristic selects the variable with the fewest remaining legal values. The Degree heuristic prioritizes variables that constrain the largest number of other variables.
Combining these techniques can reduce unnecessary exploration and improve the efficiency of timetable generation.
For very large institutions, the system can also use decomposition or optimization techniques to handle groups of related examinations.
Conclusion: CSP with backtracking, forward checking, and suitable variable-selection heuristics provides an effective method for generating valid examination timetables.

Question 5: Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. Explain how Minimax models the problem.
Minimax is a decision-making algorithm used in competitive games. It assumes that one player tries to maximize the score while the opponent tries to minimize it.
The algorithm represents possible moves as a game tree.

Working:

1. The AI generates possible moves.
2. Each move leads to a new game state.
3. The algorithm explores possible future moves.

4. It assigns evaluation scores to the resulting states.
5. At MAX nodes, it selects the highest score.
6. At MIN nodes, it selects the lowest score.
7. The AI chooses the move that provides the best outcome against an optimal opponent.

Example:

Suppose the AI has two possible moves.

- Move A has possible evaluation scores of 3 and 5.
- Move B has possible evaluation scores of 2 and 8.

Assuming the opponent chooses the minimum score:

$$A = \min(3, 5) = 3$$
$$B = \min(2, 8) = 2$$

The AI selects Move A because:

$$\max(3, 2) = 3$$

Therefore, Move A provides the better worst-case outcome.

ii. Analyze the computational challenges of large game trees.

Large game trees create several computational challenges.

1. High branching factor

Each game state may contain many possible moves.

2. Large search depth

Exploring more future moves increases the number of states that must be evaluated.

3. Time limitations

The AI must make decisions within strict time limits.

4. Memory requirements

Storing and processing large numbers of game states can consume significant memory.

5. Real-time performance

The AI must balance the quality of its decisions with response speed.

For a branching factor band search depth d, basic Minimax has a time complexity of approximately:

$$O(b^d)$$

This makes exhaustive search expensive for large games.

iii. Explain how Alpha-Beta pruning improves efficiency.

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- **Alpha (α)**: The best score MAX can guarantee so far.
- **Beta (β)**: The best score MIN can guarantee so far.

Pruning occurs when:

$$\alpha \geq \beta$$

Example:

Suppose MAX has already found a move with a value of 5.

While exploring another move, MIN finds an outcome with a value of 3. If MIN can force that outcome, the remaining branches under that MIN node cannot make the alternative move better than MAX's existing choice.

Therefore, those branches can be skipped when the bounds establish that they cannot change the final decision.

Advantages:

- Reduces the number of game states evaluated.
- Allows deeper searches within the same time limit.
- Produces the same optimal result as Minimax when applied correctly.
- Improves performance when promising moves are explored first.

In the best case, with ideal move ordering, its time complexity can approach:

$$O(b^{d/2})$$

In the worst case, its complexity remains:

$$O(b^{d})$$

iv. Design an evaluation function and justify the factors considered.

An evaluation function estimates how favorable a game state is for the AI.

A general evaluation function for a strategy game is:

$$E(s) = w_1 M + w_2 R + w_3 T + w_4 O - w_5 D$$

Where:

- M= military strength.
- R= available resources.
- T= territory controlled.
- O= strategic objectives achieved.
- D= damage or losses suffered.
- w_1, w_2, w_3, w_4, w_5= weights assigned to each factor.

Justification:

1. **Military strength:** Measures the ability to defend and attack.
2. **Resources:** Indicates the ability to produce units and maintain operations.
3. **Territory:** Measures control over strategically important areas.
4. **Objectives:** Rewards progress towards winning the game.
5. **Damage and losses:** Penalizes unfavorable exchanges and unnecessary losses.

The weights should be adjusted according to the game's rules and winning conditions.

v. Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Iterative Deepening Minimax with Alpha-Beta Pruning and a heuristic evaluation function.

Working:

1. Begin searching to a small depth.
2. Evaluate the game states.
3. Increase the search depth gradually.
4. Use Alpha-Beta pruning to eliminate unnecessary branches.
5. Apply move ordering to explore promising moves first.
6. Stop when the available time limit is reached.
7. Return the best move found in the last completed search.

Advantages:

- Makes decisions within a controlled time limit.
- Explores deeper game trees efficiently.
- Produces optimal decisions for the searched tree when evaluated exactly.
- Uses heuristic scores to evaluate states beyond the search depth.
- Supports a practical balance between decision quality and speed.

Conclusion: Iterative Deepening Minimax with Alpha-Beta Pruning is suitable for real-time strategy games because it reduces unnecessary computation while allowing the AI to make strong decisions within limited time.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand